杂题选讲

wzj52501

Peking University

2019 年 4 月 5 日

前言

- 若无其他说明，所有例题的时间限制均为 $2s$，空间限制均为 $512MB$。

例题 1

游戏（SDOI2016）

Alice 和 Bob 在玩一个游戏。

游戏在一棵有 n 个点的树上进行，树边带权且非负。最初，每个点上都只有一个数字，那个数字是 12345678912345678。

有时，Alice 会选择一条从 s 到 t 的路径，在这条路径上的每一个点上都添加一个数字：对于路径上的一个点 r，若 r 与 s 的距离是 dis，那么 Alice 在点 r 上添加的数字是 $a \times dis + b$。

有时，Bob 会选择一条从 s 到 t 的路径。他需要先从这条路径上选择一个点，再从那个点上选择一个数字。Bob 选择的数字越小越好，但大量的数字让 Bob 眼花缭乱。Bob 需要你帮他找出他能够选择的最小的数字。

$n, q \leq 10^5, |a| \leq 10000, |b| \leq 10^9$

李超树

- 首先进行树链剖分，那么我们只要解决序列的问题就可以在时间复杂度乘一个 $O(\log n)$ 的代价下解决树上问题了。

李超树

- 首先进行树链剖分，那么我们只要解决序列的问题就可以在时间复杂度乘一个 $O(\log n)$ 的代价下解决树上问题了。
- 经过转化后问题可以变成如下的简单形式：
  1. 在区间 $[l, r]$ 内添加一条直线 $y = k \times x + b$。
  2. 询问区间 $[l, r]$ 内所有直线的最小值。

李超树

- 首先进行树链剖分，那么我们只要解决序列的问题就可以在时间复杂度乘一个 $O(\log n)$ 的代价下解决树上问题了。
- 经过转化后问题可以变成如下的简单形式：
  1. 在区间 $[l, r]$ 内添加一条直线 $y = k \times x + b$。
  2. 询问区间 $[l, r]$ 内所有直线的最小值。
- 考虑使用线段树来维护这些直线，用懒标记在每个节点上标记一条覆盖整个区间的直线，表示可能作为答案的候选直线。

李超树

- 首先进行树链剖分，那么我们只要解决序列的问题就可以在时间复杂度乘一个 $O(\log n)$ 的代价下解决树上问题了。
- 经过转化后问题可以变成如下的简单形式：
  1. 在区间 $[l, r]$ 内添加一条直线 $y = k \times x + b$。
  2. 询问区间 $[l, r]$ 内所有直线的最小值。
- 考虑使用线段树来维护这些直线，用懒标记在每个节点上标记一条覆盖整个区间的直线，表示可能作为答案的候选直线。
- 当一个线段树节点 $[l, r]$ 上已经标记了一条直线，而我们又在上面添加了一条新直线，我们应该怎么维护这两条直线的并呢？

- 分别计算出两条直线左右端点的值：如果存在一个直线整体在另一条直线上的情况，直接保留最优直线即可。否则肯定存在一个交点 x，满足 $[l, x]$ 区间内直线 1 更优，而 $[x, r]$ 区间内直线 2 更优。计算两条直线在中间端点处的函数值，在这个节点保留最优区间长的一条直线，把另一条直线向下传递即可。

- 分别计算出两条直线左右端点的值：如果存在一个直线整体在另一条直线上的情况，直接保留最优直线即可。否则肯定存在一个交点 x，满足 $[l, x]$ 区间内直线 1 更优，而 $[x, r]$ 区间内直线 2 更优。计算两条直线在中间端点处的函数值，在这个节点保留最优区间长的一条直线，把另一条直线向下传递即可。
- 对于每个修改操作，$[l, r]$ 区间会拆成 $O(\log n)$ 个线段树节点，每个节点因为每次下传区间长度至少减半，所以最多传递 $O(\log n)$ 次，时间复杂度为 $O(\log^2 n)$。

- 分别计算出两条直线左右端点的值：如果存在一个直线整体在另一条直线上的情况，直接保留最优直线即可。否则肯定存在一个交点 x，满足 $[l, x]$ 区间内直线 1 更优，而 $[x, r]$ 区间内直线 2 更优。计算两条直线在中间端点处的函数值，在这个节点保留最优区间长的一条直线，把另一条直线向下传递即可。
- 对于每个修改操作，$[l, r]$ 区间会拆成 $O(\log n)$ 个线段树节点，每个节点因为每次下传区间长度至少减半，所以最多传递 $O(\log n)$ 次，时间复杂度为 $O(\log^2 n)$。
- 对于每个询问操作，每个结点维护一下子树内直线最小值，查询时在每个经过的线段树节点和被完整包含的结点上查询即可，时间复杂度为 $O(\log n)$。

- 分别计算出两条直线左右端点的值：如果存在一个直线整体在另一条直线上的情况，直接保留最优直线即可。否则肯定存在一个交点 x，满足 $[l, x]$ 区间内直线 1 更优，而 $[x, r]$ 区间内直线 2 更优。计算两条直线在中间端点处的函数值，在这个节点保留最优区间长的一条直线，把另一条直线向下传递即可。
- 对于每个修改操作，$[l, r]$ 区间会拆成 $O(\log n)$ 个线段树节点，每个节点因为每次下传区间长度至少减半，所以最多传递 $O(\log n)$ 次，时间复杂度为 $O(\log^2 n)$。
- 对于每个询问操作，每个结点维护一下子树内直线最小值，查询时在每个经过的线段树节点和被完整包含的结点上查询即可，时间复杂度为 $O(\log n)$。
- 总时间复杂度为 $O(n + m \log^3 n)$。

例题 2

Rikka with Sequence(2017 Multi-University Training Contest)

给定一个长度为 n 的序列 A，保留初始备份 $B(B_i = A_i)$，要求执行 m 次操作：

1. 给出 l、r，求和 $\sum_{i=l}^{r} A_i$。
2. 给出 l、r、k，执行 C++ 语句：for(int i=l;i<=r;i++) A[i]=A[i-k]。
3. 给出 l、r，执行 C++ 语句：for(int i=l;i<=r;i++) A[i]=B[i]。

$n, m \leq 2 \times 10^5, 1 \leq k < l, |A_i| \leq 10^9$。

时间限制 $4s$，空间限制 $64MB$。

可持久化 Treap 的空间优化技巧

- 如果忽视空间限制，很容易想到使用可持久化 Treap 解决该问题。

可持久化 Treap 的空间优化技巧

- 如果忽视空间限制，很容易想到使用可持久化 Treap 解决该问题。
- 维护节点内权值和与子树大小信息，可以通过 Treap 的分裂与合并快速提取区间信息支持操作 1、3。使用类似快速幂的算法可以在 $O(\log^2 n)$ 或 $O(\log n)$ 的时间复杂度内支持操作 2。

可持久化 Treap 的空间优化技巧

- 如果忽视空间限制，很容易想到使用可持久化 Treap 解决该问题。
- 维护节点内权值和与子树大小信息，可以通过 Treap 的分裂与合并快速提取区间信息支持操作 1、3。使用类似快速幂的算法可以在 $O(\log^2 n)$ 或 $O(\log n)$ 的时间复杂度内支持操作 2。
- 可惜可持久化数据结构的空间复杂度均与时间复杂度同阶，面对如此苛刻的空间限制只能望洋兴叹。

可持久化 Treap 的空间优化技巧

- 如果忽视空间限制，很容易想到使用可持久化 Treap 解决该问题。
- 维护节点内权值和与子树大小信息，可以通过 Treap 的分裂与合并快速提取区间信息支持操作 1、3。使用类似快速幂的算法可以在 $O(\log^2 n)$ 或 $O(\log n)$ 的时间复杂度内支持操作 2。
- 可惜可持久化数据结构的空间复杂度均与时间复杂度同阶，面对如此苛刻的空间限制只能望洋兴叹。
- 我们注意到其实在所有操作过程中，Treap 的有效节点个数并没有变化，所以我们如果能够回收那些在可持久化中失效的节点，就可以把空间复杂度保持在线性。

- 有一种办法是通过维护一些信息自动判定节点是否失效，不过这种垃圾回收的方法实现较为繁琐且常数较大。

- 有一种办法是通过维护一些信息自动判定节点是否失效，不过这种垃圾回收的方法实现较为繁琐且常数较大。
- 另一种简单高效的方法是定期重构，考虑每完成 $\frac{m}{\log m}$ 次操作进行一次重构：把当前数列 A 全部提取，然后将可持久化 Treap 清空，最后按提取出来的数列 A 重新建立 Treap。这样一来相当于把 $O(m)$ 个操作平分成 $O(\log m)$ 组，每组的数据规模缩小了 $O(\log m)$ 级别，空间复杂度被降为 $O(\frac{m}{\log m} \times \log n) = O(m)$。

- 有一种办法是通过维护一些信息自动判定节点是否失效，不过这种垃圾回收的方法实现较为繁琐且常数较大。
- 另一种简单高效的方法是定期重构，考虑每完成 $\frac{m}{\log m}$ 次操作进行一次重构：把当前数列 A 全部提取，然后将可持久化 Treap 清空，最后按提取出来的数列 A 重新建立 Treap。这样一来相当于把 $O(m)$ 个操作平分成 $O(\log m)$ 组，每组的数据规模缩小了 $O(\log m)$ 级别，空间复杂度被降为 $O(\frac{m}{\log m} \times \log n) = O(m)$。
- 每进行一次重构的代价是 $O(n)$，所以分组的总代价为 $O(n \log m)$，并不影响时间复杂度。
- 总时间复杂度为 $O((n+m)\log n)$，总空间复杂度为 $O(n+m)$。

例题 3

资源采集

在遥远的精灵世界，精灵们在精灵圣泉沿线修筑了 n 个能量井，其中第 i 号能量井每时刻吸取能量值为 v_i，最大储存能量值为 l_i。

这 n 个能量井形成了一个有根树形结构，其中 1 号能量井是精灵圣泉的发源地，设 i 号能量井的上游父亲节点为 f_i，与父亲节点距离为 d_i。

你需要为精灵一族模拟 q 个采集过程。设初始时刻 0 时所有能量井均为空，某些时刻 t_i 你需要在 x_i 号能量井派出若干采集队，顺流而下采集所有与 x_i 号能量井距离不超过 k_i（设 x_i 号能量井到自身距离为 0）的能量井中的所有能量（能量井被采集后其中的能量将被清空，但仍会自动补充），请输出每次采集中获得的总能量值。

$1 \leq n, q \leq 52501, 1 \leq v_i, l_i, d_i, t_i, k_i \leq 10^9$ $1 \leq f_i, x_i \leq N$。

保证 t_i 严格递增。

链上问题

- 对于整棵树形成一条链的情况：

链上问题

- 对于整棵树形成一条链的情况：
- 我们考虑用平衡树维护链上每个节点上次被采摘的时间，不难发现我们维护的是一段一段相等的时间区间，而对于每次采摘都至多让不同的时间区间增多 1。

链上问题

- 对于整棵树形成一条链的情况：
- 我们考虑用平衡树维护链上每个节点上次被采摘的时间，不难发现我们维护的是一段一段相等的时间区间，而对于每次采摘都至多让不同的时间区间增多 1。
- 我们可以对于上次采摘时间相同的节点，用主席树计算这一段连续的节点产生的贡献，其中 $\lceil \frac{l_i}{v_i} \rceil \leq T$ 的节点会被填满对答案有 l_i 的贡献，其余的节点会产生 $v_i \times T$ 的贡献。以时间段为势能函数分析，每次操作最多势能加 1，而每用主席树计算一次势能 -1。

链上问题

- 对于整棵树形成一条链的情况：
- 我们考虑用平衡树维护链上每个节点上次被采摘的时间，不难发现我们维护的是一段一段相等的时间区间，而对于每次采摘都至多让不同的时间区间增多 1。
- 我们可以对于上次采摘时间相同的节点，用主席树计算这一段连续的节点产生的贡献，其中 $\lceil \frac{l_i}{v_i} \rceil \leq T$ 的节点会被填满对答案有 l_i 的贡献，其余的节点会产生 $v_i \times T$ 的贡献。以时间段为势能函数分析，每次操作最多势能加 1，而每用主席树计算一次势能 -1。
- 时间复杂度为 $O(n \log n)$。

无上限

- 对于能量泉没有容量限制的情况。

无上限

- 对于能量泉没有容量限制的情况。
- 对于每个能量泉，将 (*dfs* 序位置，深度) 作为坐标放到直角坐标系上，则每一次采摘的能量泉对应一个矩形内的所有点。

无上限

- 对于能量泉没有容量限制的情况。
- 对于每个能量泉，将 (*dfs* 序位置，深度) 作为坐标放到直角坐标系上，则每一次采摘的能量泉对应一个矩形内的所有点。
- 考虑用 *KD* 树维护这些点，每次操作期望影响 $\sqrt{n}$ 个单点与 $\sqrt{n}$ 个 *KD* 树上的矩形，可以很容易通过打标记之类的方式维护答案。

无上限

- 对于能量泉没有容量限制的情况。
- 对于每个能量泉，将 $(dfs$ 序位置，深度$)$ 作为坐标放到直角坐标系上，则每一次采摘的能量泉对应一个矩形内的所有点。
- 考虑用 KD 树维护这些点，每次操作期望影响 $\sqrt{n}$ 个单点与 $\sqrt{n}$ 个 KD 树上的矩形，可以很容易通过打标记之类的方式维护答案。
- 时间复杂度为 $O(n \log n + q\sqrt{n})$。

Full

- 结合两部分算法的思想，每次操作的时候先找到这些单点和矩形，然后从每个矩形继续 *dfs* 到有标记的小矩形处，用与算法二类似的方式可以 $O(\log n)$ 计算贡献。
- 这里需要用到可持久化线段树或平衡树的合并。

Full

- 结合两部分算法的思想，每次操作的时候先找到这些单点和矩形，然后从每个矩形继续 *dfs* 到有标记的小矩形处，用与算法二类似的方式可以 $O(\log n)$ 计算贡献。
- 这里需要用到可持久化线段树或平衡树的合并。
- 以当前有标记的矩形个数进行势能分析，每次操作期望增加 $\sqrt{n}$ 的势能，每次计算贡献 -1 势能。
- 时间复杂度为 $O(n \log n + q\sqrt{n} \log n)$。

Break

Q1

A、B、C 三个同学周末决定去聚餐，但当他们准备点餐的时候却发现这顿饭已经点好并付款了。他们知道有可能是他们其中一个人提前付款，也可能是别人请的客。

现在每个人都尊重其他两个人匿名请客的权利，但他们想知道是不是三个人外的人请的客。请设计一个方案，使得三个人能够通过某些信息的交互知道是否有其他人请客，且若是三个人中的某人买单，他们无法从信息中推断出是谁。

A1

- 一种可行的方案是：
- Step1、每两个人抛一枚硬币，第三个人不知道这次抛掷的结果。

A1

- 一种可行的方案是：
- Step1、每两个人抛一枚硬币，第三个人不知道这次抛掷的结果。
- Step2、每个人宣布他看到的两枚硬币结果是否一致。若付款的是这个人，则他会说相反的结果。

A1

- 一种可行的方案是：
- Step1、每两个人抛一枚硬币，第三个人不知道这次抛掷的结果。
- Step2、每个人宣布他看到的两枚硬币结果是否一致。若付款的是这个人，则他会说相反的结果。
- Step3、若声称不同人有奇数个，则说明三人中的一人买单；否则说明有其他人请客。

例题 4

线路规划

给定一棵 n 个结点的无根树，你需要根据 m 条线路重构一棵生成树。其中第 i 条线路可以用四元组 (x_i, y_i, k_i, w_i) 来表达：

- 你可以在 x_i 和 y_i、x_i 的父亲和 y_i 的父亲、……、x_i 的 $k_i - 1$ 级祖先和 y_i 的 $k_i - 1$ 级祖先之间任意添加边，每一条的费用都是 w_i。

求总费用之和最小的方案，保证题目有解且 x_i 和 y_i 均至少有 $k_i - 1$ 个祖先。

$n, m \leq 152501$

倍增并查集

- 先从简单情况入手，考虑解决链上问题。

倍增并查集

- 先从简单情况入手，考虑解决链上问题。
- 按照 Kruskal 算法先将边权进行排序，那么我们需要做的其实是这样一件事情：
- 将区间 $[l_1, r_1]$ 里每一个结点向区间 $[l_2, r_2]$ 里对应的结点连边。

倍增并查集

- 先从简单情况入手，考虑解决链上问题。
- 按照 Kruskal 算法先将边权进行排序，那么我们需要做的其实是这样一件事情：
- 将区间 $[l_1, r_1]$ 里每一个结点向区间 $[l_2, r_2]$ 里对应的结点连边。
- 考虑使用倍增 ST 表的思想，维护 $\log n$ 层并查集。其中第 i 层的并查集 $pa_i(x)$ 维护的是以 x 为起点的长度为 2^i 的一段区间的连通信息。

cont'd

- 当我们合并一对长度为 2^i 的区间 $[x, x+2^i-1]$ 与 $[y, y+2^i-1]$ 时，先检查 $pa_i(x)$ 是否已经和 $pa_i(y)$ 连通，否则将 $pa_i(x)$ 与 $pa_i(y)$ 合并，并且暴力递归第 $i-1$ 层的 (x, y) 和 $(x+2^{i-1}, y+2^{i-1})$ 进行合并。这样的合并显然至多会发生 $O(n \log n)$ 次。

cont'd

- 当我们合并一对长度为 2^i 的区间 $[x, x+2^i-1]$ 与 $[y, y+2^i-1]$ 时，先检查 $pa_i(x)$ 是否已经和 $pa_i(y)$ 连通，否则将 $pa_i(x)$ 与 $pa_i(y)$ 合并，并且暴力递归第 $i-1$ 层的 (x, y) 和 $(x+2^{i-1}, y+2^{i-1})$ 进行合并。这样的合并显然至多会发生 $O(n \log n)$ 次。
- 这样在合并一对普通区间时可以类似 RMQ 问题将其拆成两对长度为 2 的整数次幂的区间，问题就能得到解决。时间复杂度为 $O(m + n \log n \alpha(n))$。

cont'd

- 当我们合并一对长度为 2^i 的区间 $[x, x+2^i-1]$ 与 $[y, y+2^i-1]$ 时，先检查 $pa_i(x)$ 是否已经和 $pa_i(y)$ 连通，否则将 $pa_i(x)$ 与 $pa_i(y)$ 合并，并且暴力递归第 $i-1$ 层的 (x, y) 和 $(x+2^{i-1}, y+2^{i-1})$ 进行合并。这样的合并显然至多会发生 $O(n \log n)$ 次。
- 这样在合并一对普通区间时可以类似 RMQ 问题将其拆成两对长度为 2 的整数次幂的区间，问题就能得到解决。时间复杂度为 $O(m + n \log n\alpha(n))$。
- 然后树上问题只需要使用树链剖分将两条路径分解成 $O(\log n)$ 对 DFS 序上的区间，套用链上算法即可。总时间复杂度为 $O(m \log n + n \log n\alpha(n))$。

例题 5

欧拉子图

给定一个 n 个点，m 条边无向图（可能有重边），对于这个图的边集的子集（一共有 2^m 个），如果其导出的子图的每个连通块内都存在欧拉回路，我们就把答案加上这个子图的边数的平方，答案对 998244353 取模。

$n, m \leq 152501$

- 首先欧拉子图的充要条件为每个点的度数均为偶数，设 $E(x, y)$ 为第 x 条边与第 y 条边均被选的方案，则

$$ans = \sum_{1 \le x \le m, 1 \le y \le m} E(x, y)$$

- 首先欧拉子图的充要条件为每个点的度数均为偶数，设 $E(x, y)$ 为第 x 条边与第 y 条边均被选的方案，则

$$ans = \sum_{1 \le x \le m, 1 \le y \le m} E(x, y)$$

- 一个简单的想法是使用高斯消元求出线性方程组的解数，但这样显然没有挖掘多少图的性质。

- 首先欧拉子图的充要条件为每个点的度数均为偶数，设 $E(x, y)$ 为第 x 条边与第 y 条边均被选的方案，则

$$ans = \sum_{1 \le x \le m, 1 \le y \le m} E(x, y)$$

- 一个简单的想法是使用高斯消元求出线性方程组的解数，但这样显然没有挖掘多少图的性质。
- 把每个连通分量分开考虑，不难发现对于一个 n 个点 m 条边的连通分量，欧拉子图方案数为 2^{m-n+1}。

图连通性的 Hash 维护

- 设连通块数量为 s，枚举 (x, y)，若 (x, y) 中有桥边，则必定无解。否则若删去后连通性不变，方案数为 $2^{m-n+s-2}$；若删去后连通块多一个，方案数为 $2^{m-n+s-1}$。

图连通性的 Hash 维护

- 设连通块数量为 s，枚举 (x, y)，若 (x, y) 中有桥边，则必定无解。否则若删去后连通性不变，方案数为 $2^{m-n+s-2}$；若删去后连通块多一个，方案数为 $2^{m-n+s-1}$。
- 问题转化成判断有多少组 (x, y) 删去后会改变连通性，这里有一个经典的方法：找到一棵生成树，给每条非树边随机一个权值，每条树边的权值记为覆盖它的非树边的权值异或和。这样若 x 和 y 权值相同，说明连通性发生了改变。

图连通性的 Hash 维护

- 设连通块数量为 s，枚举 (x, y)，若 (x, y) 中有桥边，则必定无解。否则若删去后连通性不变，方案数为 $2^{m-n+s-2}$；若删去后连通块多一个，方案数为 $2^{m-n+s-1}$。
- 问题转化成判断有多少组 (x, y) 删去后会改变连通性，这里有一个经典的方法：找到一棵生成树，给每条非树边随机一个权值，每条树边的权值记为覆盖它的非树边的权值异或和。这样若 x 和 y 权值相同，说明连通性发生了改变。
- 时间复杂度为 $O(m \log m)$。

route

求索

给定一棵 n 个结点的无根树，边上有权值 w_i。求一条路径满足路径上边权的最小值与路径边权之和的乘积尽量大，且这条路径上的边权的极差不能超过 lim，输出最大乘积。

$n \leq 152501, 0 \leq w_i, lim \leq 52501$

树直径的快速维护

- 命题：对于树上两个不相交的点集 s、t，设集合 s 内任意两点间路径长度最长的路径之一为 $(x_1 \to y_1)$，集合 t 内任意两点间路径长度最长的路径之一为 $(x_2 \to y_2)$。则 $s \cup t$ 集合内任意两点间路径长度最长的路径之一必定属于集合

$$\{(x_1 \to y_1), (x_2 \to y_2), (x_1 \to x_2), (x_1 \to y_2), (x_2 \to x_1), (x_2 \to y_1)\}$$

Proof

反证法，设 $s \cup t$ 内有一条边权最大的路径 $(u \to v)$ 不属于答案集合且满足其边权大于集合内任意一条路径。

Proof

反证法，设 $s \cup t$ 内有一条边权最大的路径 $(u \to v)$ 不属于答案集合且满足其边权大于集合内任意一条路径。

1. 若 u, v 同属于 s 或 t 集合，设它们同属于集合 s，则 $(u \to v)$ 路径会比 $(x_1 \to y_1)$ 优，这显然与假设相悖。

Proof

反证法，设 $s \cup t$ 内有一条边权最大的路径 $(u \to v)$ 不属于答案集合且满足其边权大于集合内任意一条路径。

1. 若 u, v 同属于 s 或 t 集合，设它们同属于集合 s，则 $(u \to v)$ 路径会比 $(x_1 \to y_1)$ 优，这显然与假设相悖。
2. 若 u, v 不同属于 s 或 t 集合，设 u 属于集合 s。

Proof

反证法，设 $s \cup t$ 内有一条边权最大的路径 $(u \to v)$ 不属于答案集合且满足其边权大于集合内任意一条路径。

1. 若 u, v 同属于 s 或 t 集合，设它们同属于集合 s，则 $(u \to v)$ 路径会比 $(x_1 \to y_1)$ 优，这显然与假设相悖。
2. 若 u, v 不同属于 s 或 t 集合，设 u 属于集合 s。
    1. 若 x_1, y_1 任意一个在路径 $(u \to v)$ 上，设 x_1 在路径上，我们取 $(y_1 \to v)$ 会优于路径 $(u \to v)$。

Proof

反证法，设 $s \cup t$ 内有一条边权最大的路径 $(u \to v)$ 不属于答案集合且满足其边权大于集合内任意一条路径。

1. 若 u, v 同属于 s 或 t 集合，设它们同属于集合 s，则 $(u \to v)$ 路径会比 $(x_1 \to y_1)$ 优，这显然与假设相悖。
2. 若 u, v 不同属于 s 或 t 集合，设 u 属于集合 s。
    1. 若 x_1, y_1 任意一个在路径 $(u \to v)$ 上，设 x_1 在路径上，我们取 $(y_1 \to v)$ 会优于路径 $(u \to v)$。
    2. 若 x_1, y_1 均不在路径 $(u \to v)$ 上，则路径 $(y_1 \to v)$ 和 $(x_1 \to v)$ 中肯定存在一个比路径 $(u \to v)$ 更优。

Proof

反证法，设 $s \cup t$ 内有一条边权最大的路径 $(u \to v)$ 不属于答案集合且满足其边权大于集合内任意一条路径。

1. 若 u, v 同属于 s 或 t 集合，设它们同属于集合 s，则 $(u \to v)$ 路径会比 $(x_1 \to y_1)$ 优，这显然与假设相悖。
2. 若 u, v 不同属于 s 或 t 集合，设 u 属于集合 s。
    1. 若 x_1, y_1 任意一个在路径 $(u \to v)$ 上，设 x_1 在路径上，我们取 $(y_1 \to v)$ 会优于路径 $(u \to v)$。
    2. 若 x_1, y_1 均不在路径 $(u \to v)$ 上，则路径 $(y_1 \to v)$ 和 $(x_1 \to v)$ 中肯定存在一个比路径 $(u \to v)$ 更优。

Q.E.D

cont'd

- 从大到小枚举路径上最小的边权，有了以上的结论，可以很容易地在加入边的同时维护直径，但仍不容易删除边。

cont'd

- 从大到小枚举路径上最小的边权，有了以上的结论，可以很容易地在加入边的同时维护直径，但仍不容易删除边。
- 考虑类似动态图连通性的方法进行线段树分治，同时用按秩合并的并查集（为了撤销操作）维护答案。

cont'd

- 从大到小枚举路径上最小的边权，有了以上的结论，可以很容易地在加入边的同时维护直径，但仍不容易删除边。
- 考虑类似动态图连通性的方法进行线段树分治，同时用按秩合并的并查集（为了撤销操作）维护答案。
- 时间复杂度为 $O(n \log^2 n)$。

THX

Q & A